ALGORITHM: N-Puzzle Backtracking Solution

```
// Global variables
GOAL_STATE = [[1,2,3], [4,5,6], [7,8,0]]  // for 8-puzzle
visited_states = empty set
solution_path = empty list

FUNCTION solve_npuzzle(initial_state):
    visited_states.clear()
    solution_path.clear()

    IF backtrack(initial_state, solution_path):
        RETURN solution_path
    ELSE:
        RETURN "No solution exists"

FUNCTION backtrack(current_state, path):
    // Base case 1: Check if goal state reached
    IF current_state == GOAL_STATE:
        RETURN true

    // Base case 2: Check if state already visited (cycle detection)
    IF current_state IN visited_states:
        RETURN false

    // Mark current state as visited
    visited_states.add(current_state)

    // Find position of empty space (0)
    empty_row, empty_col = find_empty_position(current_state)

    // Generate all possible moves
    possible_moves = [UP, DOWN, LEFT, RIGHT]

    FOR each move IN possible_moves:
        // Calculate new position after move
        new_row, new_col = calculate_new_position(empty_row, empty_col, move)

        // Check move validity
        IF is_valid_move(new_row, new_col, current_state):
            // Make decision: Apply the move
            new_state = apply_move(current_state, empty_row, empty_col, new_row, new_col)
            path.append(move)
```

```
            // Recursive exploration
            IF backtrack(new_state, path):
                RETURN true  // Solution found

            // Backtrack: Undo the move
            path.remove_last()

    // Remove current state from visited (allow revisiting from different paths)
    visited_states.remove(current_state)
    RETURN false  // No solution found from this state

FUNCTION is_valid_move(row, col, state):
    // Check boundary conditions
    IF row < 0 OR row >= state.rows OR col < 0 OR col >= state.cols:
        RETURN false
    RETURN true

FUNCTION apply_move(state, empty_row, empty_col, new_row, new_col):
    new_state = copy(state)
    // Swap empty space with target tile
    new_state[empty_row][empty_col] = state[new_row][new_col]
    new_state[new_row][new_col] = 0
    RETURN new_state

FUNCTION find_empty_position(state):
    FOR i = 0 TO state.rows-1:
        FOR j = 0 TO state.cols-1:
            IF state[i][j] == 0:
                RETURN i, j

FUNCTION calculate_new_position(row, col, move):
    SWITCH move:
        CASE UP: RETURN row-1, col
        CASE DOWN: RETURN row+1, col
        CASE LEFT: RETURN row, col-1
        CASE RIGHT: RETURN row, col+1

// Main execution
initial_state = [[2,8,3], [1,6,4], [7,0,5]]  // Example scrambled state
result = solve_npuzzle(initial_state)
PRINT result
```